CS201 Lab 7

Data Structures and Algorithms

Shreya Agarwal Rachit Nimavat

Spring 2026

This lab builds directly upon your `IntVector` library from previous labs. You will need your `vector.h`, `vector.c`, and `Makefile`. If your implementation of `IntVector` was buggy, you may start fresh by downloading a copy from the CS201 labs Github.

Part 1: k-Best Trades

Recall the best trade problem from last lab. Now, you are allowed to make at most `k` buy-sell transactions. However, you cannot engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again). Given an array `int price[n]` of daily stock prices and an integer `k`, find the maximum profit you could have achieved by making at most `k` non-overlapping buy-sell transactions.

Example: Input: $[100, 80, 50, 60, 70, 40, 90]$ and $k = 3$. Output: 70 by buying @50 and selling @70 followed by buying @40 and selling @90. Note that we only made 2 transactions even though we were allowed 3.

Design an algorithm that runs in $O(n^2k)$ time[1] [2]. Improve it to $O(nk)$ time[3]. After coding your solution for this problem, submit it on Codeforces.

Towards the end of this course, we will improve our solution to $O(n \log n)$ time[4], and for extra extra challenge, even $O(n)$ time.

- Traverse the array and identify every pair of local minima (valley) and local maxima (peak).
- If you buy at every valley and sell at every peak, you get the maximum possible profit (effectively).
- Count these pairs. If the count , you are done! The answer is simply the sum of all these profits.

Hint 2: Having "Too Many" Trades The problem arises when the number of peak-valley pairs (let's say) is greater than . You now have potential transactions, but you are only allowed . You need to iteratively reduce the number of transactions by . To do this efficiently, you must either:

[1] Use a 2D array `dp` where `dp[i][j]` represents the maximum profit achievable with `i` transactions up to day `j`. How'd you fill this table?

[2] The base case is $dp[0][j] = 0$ for all j and $dp[i][0] = 0$ for all i. Update this table based on previous computations:

$$dp[i][j] = \max(dp[i][j-1], \max_{k<j}(dp[i-1][k] + price[j] - price[k]))$$

Here, the first term represents not making a transaction on day `j`, while the second term considers all possible previous days `k` to buy before selling on day `j`.

[3] Optimize the inner loop by maintaining a variable that tracks the maximum value of `dp[i-1][k] - price[k]` as you iterate through the days. This allows you to compute `dp[i][j]` in constant time.

[4] Uses Heaps to efficiently manage and select the most profitable trades. We will revisit this towards the end of the course! For a sneak peek, look at the hint below.[^hint-nlogn2]

1. **Delete** a low-profit transaction entirely.
2. **Merge** two adjacent transactions into one.

Hint 3: The Cost of Merging Consider two adjacent transactions:

- Transaction A: Buy at , Sell at
- Transaction B: Buy at , Sell at
- (Where)

If you merge them, you essentially skip the "dip" between and . The new transaction is Buy at , Sell at .

- **Cost of Merge:** The profit you "give up" is .
- **Cost of Deletion:** The profit you "give up" is .

Hint 4: The Data Structure You need to repeatedly find the *minimum* cost to reduce the transaction count (either by merging or deleting) until you reach transactions.

- Which data structure allows you to retrieve and remove the minimum element in time?
- Push all potential "costs" (losses) into this structure. Repeatedly pop the smallest loss and adjust your total profit accordingly.

Hint 5: Linking it Together When you merge two segments, they become one new segment. Be careful—this new segment might now be adjacent to *other* segments, creating new merge possibilities. A Doubly Linked List is useful here to keep track of which segments are neighbors in time, while the Heap handles the costs. –>